

孙武侦查官 · 确定性侦查推演内核

技术方案书（内部技术设计文档）

文档性质	内部技术设计文档（给研发 / 架构 / 代码审计）
依据版本	D:\dev\inves_duckdb 仓库源码逐模块核对
篇幅	8 章 · 约 42,074 字
生成日期	2026-09-11
技术栈	Python 3.12+ · DuckDB · LadybugDB（可选） · FastAPI · Vue 3

本文档全部结论以源码为唯一事实来源，未引用既有 md / txt 文档结论。
凡与既有文档冲突之处，一律以代码为准并在第 8 章记录。

孙武侦查官 · 确定性侦查推演内核

技术方案书（内部技术设计文档）

本文档基于 `D:\dev\inves_duckdb` 仓库源码逐模块核对撰写，共 8 章、约 42,074 字。

事实来源：源码本身（未引用既有 md/txt 文档结论）。凡与既有文档冲突之处，一律以代码为准并在第 8 章记录。

生成日期：2026-09-11

目录

- 第 1 章 项目定位与设计目标 （4,447 字 / 目标 4,500 字）
- 第 2 章 总体架构设计 （5,888 字 / 目标 6,000 字）
- 第 3 章 语义层: Ontology 编译内核 （6,506 字 / 目标 6,500 字）
- 第 4 章 数据接入与治理 （4,462 字 / 目标 5,000 字）
- 第 5 章 规则引擎与 Function 计算层 （5,919 字 / 目标 6,000 字）
- 第 6 章 线索生命周期、处置与写路径 （4,860 字 / 目标 5,000 字）
- 第 7 章 安全、权限与可观测性 （5,481 字 / 目标 5,500 字）
- 第 8 章 质量保障、部署与演进路线 （4,511 字 / 目标 4,500 字）

第 1 章 项目定位与设计目标

「孙武侦查官」是一个确定性侦查推演内核（编程语言 Python 3.12+，存储 DuckDB，可选图库 LadybugDB，前端 Vue 3 + TypeScript），部署形态纯离线、零 LLM 依赖、零 API Key，可运行于物理隔离网络。本章界定项目要解决的问题、系统的能力边界、用代码强制守住的三条禁令、以及贯穿全系统的六条设计原则，并给出阅读本文档所需的术语表与读者路径。本章不展开模块实现细节，那些留待第 2、3、5 章；本章只回答“我们到底在造什么、为什么这么造、哪些事我们故意不做”。

1.1 问题域：调查分析工作的三类固有困境

侦查研判在很长一段时间里是一种建立在个人经验之上的手艺。资深侦查员凭借对资金流向、通话规律、工商关联、轨迹重合的直觉，把分散在多张原始表里的线索拼成一条可追查的链条。这种手艺在实战中有效，但它有三个从结构上无法回避的困境，而这三个困境恰好也是机器能够、也应当补位的地方。

第一类困境是经验手艺不可复现。同一批数据交给两位不同的分析师，结论可能大相径庭；同一位分析师在周一和周五面对同样的材料，也可能给出不一样的判断。问题不在于人，而在于“为什么这么判”这件事没有被显式地表达出来——判据藏在人的脑子里，没有落在可重放的指令上。当一份研判报告需要向上级解释、需要被复核、需要在半年后被另案引用时，这种不可复现性就变成了追责与合规上的硬伤。本项目的核心命题，就是把“查什么、怎么判定、阈值多少”从人的脑内状态，外化成一组写在 `ontology/<pack>/*.json` 里的声明，使同样的输入永远得到同样的输出，使任何一次推演都能被完整回放。

第二类困境是多源异构数据无法对齐。真实调查材料来自银行流水、通话记录、招投标档案、工商登记、轨迹出行、公开 OSINT、举报材料七类原始来源，它们的字段命名、粒度、主键、时间口径彼此不同，且大多以中文业务表名和异构字段形态存在。在没有统一语义层的情况下，每新接入一种数据源，工程上就要重写一批针对该表结构的查询与连接逻辑，形成 $N \times M$ 的对账成本。项目以 DuckDB 温层承载中文业务冷表，并在其上编译出 `obj_*/lnk_*` 语义层（`objects.json / links.json / bindings.json`），把“数据是什么”与“数据从哪来”解耦，使规则只面向稳定的语义对象编写，而非面向每一张原始表重写。

第三类困境是结论无法追溯到原始行。调查工作的产物最终要能进法庭、能经得起质证，这意味着每条线索都必须能回答“你凭哪一行、哪一张表、哪个版本的数据得出这个结论”。如果一条线索只能说“系统算出来的”，它就不具备可采性。本项目因此把溯源做成一等公民：Function 不返回原始明细，只返回聚合结果与内容寻址的溯源 ID（`core/row_uri.py` 采用 `dataset@version#partition/rowid` 格式），需要时由 `resolve_row_uri` 回捞原始行。溯源 ID 绑定了数据集、版本与分区，使任意一条线索都能沿审计哈希链回到它诞生时刻的原始数据快照。

多源无法对齐的代价值得单独量化。项目面对的七类原始来源里，银行流水与轨迹出行以明细行粒度存在、工商登记以主体粒度存在、招投标档案以项目粒度存在、通话记录以“主叫-被叫-时刻”三元组存在，彼此没有共享主键。在没有语义层时，要把“张三在某次中标后与某账户的大额往来”连成一条链，需要手写跨表 JOIN、做人名与组织名归一（`core/entity.py` 的 `normalize_org_name / OrganizationResolver`）、做时间窗对齐，而这套逻辑对每一对新数据源都要重做一遍，这就是 $N \times M$ 对账成本的来源。语义层把这一成本一次性沉到 `bindings.json` 的管道层：类型层（`objects.json / links.json`）保持稳定，新数据源只需补一份绑定声明。

这三类困境不是孤立的。经验不可复现的根因之一是判定没被显式化；多源无法对齐放大了显式化的成本；而溯源缺失则让前两者的产出失去法律效力。项目后续所有架构决策，都可以视作对这三个根问题的回应。

1.2 系统定位与能力边界

本系统定位为“确定性侦查推演内核”，它处于调查流程的计算与提示环节，而不是决策与处置环节。更具体地说，它把多源异构原始数据编译成一套语义层，在其上执行声明式规则，产出带溯源 ID 的线索，再经去重、优先级排序、五间交叉升格，进入人工处置看板。整个链路在核心路径上不依赖任何大模型；LLM 仅作为可选增强层，承担草案生成、结果解释、字段对齐等辅助任务，且带一键降级开关与影子模式（REQ-040）。

系统的能力边界由一条纪律性命题划清：机器只做计算与提示，绝不作定性结论，绝不置“已立案”。这条边界不是写在文档里的软约定，而是在代码里由状态机与操作者校验双重强制——“已立案”被硬编码进 `core/access.py` 的 `HUMAN_ONLY_STATUSES = frozenset({"已立案"})`，任何非人操作者发起的状态迁移都会被拒绝；线索从“已固证”升格到“已立案”必须携带 `legal_basis` 且仅可由具名的人操作者触发。这样做的意图很明确：定性结论与立案决定属于人类执法权限，系统只负责把证据链算清楚、把可疑点提示出来、把每一步的依据留痕，最终是否定性、是否立案交由人判断。

在“不做什么”之外，同样重要的是“不替代什么”。系统不替代数据接入前的合法取证，不替代人工对证据真实性的质证，不替代司法程序里的任何一环。它能降低的是经验不可复现带来的不确定性、数据不对齐带来的工程成本、溯源缺失带来的合规风险。明确边界的意义在于防止系统被误用为“自动定罪”工具——这是设计上必须主动拒绝的场景。

系统把“多源 → 语义层 → 规则 → 线索 → 处置”串成一条确定性链路，并沿五技能流水线推进：庙算（MiaoSuan 假设编排）→ 知己（覆盖盘点）→ 虚实（R1-R6 规则执行）→ 奇正（时间窗碰撞等对抗性判定）→ 用间（五间交叉升格）。其中“用间”对应项目最关键的升格逻辑：单源线索只是观察，两源交叉成为线索，三源以上升级为“可立案依据候选”——这条映射硬编码不可配（仅间类名称可配），因为升格阈值一旦可被随意调整，就动摇了“线索等级”这一判定本身的客观性。最终产物进入 `DisposalBoard`（处置看板），由人完成“已固证→已立案”的闭环。

1.3 三条禁令及其代码强制点

三条禁令是前文能力边界的代码化表达。它们的共同特征是“用代码强制，而非用规范劝导”：违反禁令的调用会在运行时被拒绝，且其中两条还有静态扫描进入 CI 作为不可绕过的红线。

第一条禁令是不自己写业务 SQL。内核禁止任何业务代码直接查询七张原始业务表。强制点在 `core/store.py` 维护的 `_FORBIDDEN_TABLES = (银行流水, 通话记录, 招投标档案, 工商信息, 轨迹出行, 公开OSINT, 举报材料)`，门面层 `Store` 在 `touches_forbidden_table()` 中命中即抛 `DirectSourceAccessError`。此外 `scripts/audit_straight_sql.py` 做静态扫描，CI 测试组 `audit` 强制通过，使直查代码无法合入主干。绕过通道仅剩 `unsafe=True`，且必须带 `operator + reason` 并落入 `meta_unsafe_query` 审计表——即任何绕过都是显式、具名、可追溯的例外，而非静默的后门。这条禁令的代价是：所有面向原始数据的查询都必须先经语义层编译，接入新数据源有前置成本；收益是七张异构原始表永远不会被规则逻辑直接耦合，溯源与重建因此成为可能。

第二条禁令是不把原始明细搬进上下文。Function 只返回聚合结果与溯源 ID，绝不下发逐行原始明细，以防止敏感个人信息在推理链路中被无意义地扩散。强制点在 `core/row_uri.py` 的内容寻址机制：原始行由 `make_row_uri` 以 `dataset@version#partition/rowid` 编码，明细归档进 `row_archive / row_build_index` 表，需要回看时由 `resolve_row_uri / snapshot_source_rows` 精确回捞。换句话说，上下文里流动的是“指向数据的指针 + 聚合结论”，原始数据留在受控存储里按需取用。代价是回看原始行需要一次额外解析；收益是上下文规模可控、敏感字段不漫游、且每一份被引用的明细都能被精确锚定到版本化快照。

第三条禁令是不下定性结论、不置“已立案”。强制点有两处：其一是 `core/access.py` 的 `HUMAN_ONLY_STATUSES`，把“已立案”钉死为仅人类可写入的终态；其二是 `core/action_executor.py` 的 `FORBIDDEN_OPERATORS = {system, ai, assistant, model, bot, auto, llm}`，凡以这些身份发起写操作一律 `ValueError`。在 MCP 边界上，`clue_transition` 工具还额外拦截 `agent:` 前缀操作者，堵住“以 Agent 名义改状态”的口子。代价是系统永远无法“自

已结案”，必须由具名的人闭环；收益是定性权与立案权始终留在人类执法者手中，系统输出始终可被定性为“计算与提示”。

三条禁令彼此咬合：第一条禁止直查原始表，使所有计算流经语义层，为第二条的溯源 ID 提供稳定的命名空间；第二条限制原始明细进入上下文，使敏感数据不漫游，为第三条的“只算不判”减少越权风险；第三条把定性权锁回人类，使前两条的克制不至于在出口处被一笔勾销。它们共同构成系统的“不可自作主张”底座。

1.4 六条设计原则

六条设计原则是前三条禁令的推广，它们定义了系统在每个架构岔路口上的默认选择。每条原则都对应一个具体问题，也都附带可计算的代价。

确定性优先于智能。核心链路（语义编译、规则执行、线索去重排序、写路径）不依赖大模型；LLM 仅作为可选增强层承担草案/解释/对齐，且带一键降级开关与影子模式（REQ-040）。具体机制上，`core/llm/fallback.py` 提供一键降级并定义 `LLM_OWNED_TABLES`，降级后 LLM 相关写能力整体关闭；解释生成 `core/llm/explain.py` 内置 `_QUALITATIVE_WORDS` 定性词黑名单，防止增强层越界输出定性结论；离线环境下 `core/llm/llm_client.py` 的 `fake_invoke` 注入使链路在无 key 时仍可跑通。它解决的问题是推理结果可复现、可在物理隔离环境运行；代价是系统不擅长开放域语义理解，需要靠声明式规则覆盖能力，规则未覆盖的盲区只能交给人或降级提示。

声明是数据，实现是代码。“查什么、怎么判定、谁能看”全部写在 `ontology/<pack>/*.json` 声明文件里，Python 只提供编译器、执行器、原子能力。解决的问题是业务变更不触发代码重发、规则可被审计可回放；代价是引入一套声明式 DSL 与装载顺序的硬依赖，学习曲线与维护面转移到声明层。

类型层与管道层分离。`objects.json` / `links.json` 定义“是什么”（类型层），`bindings.json` 定义“数据从哪来”（管道层）。解决的问题是多源接入时无需为每种源重写类型定义，且类型稳定而管道可替换；代价是两个层必须保持契约一致，装载顺序有硬依赖（类型层先于管道层）。

失败要留痕，不许静默。零命中四分类、脏值降级计数、结构降级、派发失败全部进入 `run_diagnostic`，与 `findings` 物理分离。解决的问题是“没查到”与“查了没中”被区分，管线故障可见可追责；代价是诊断数据规模增长，需要独立的诊断视图而非混进结果。

未声明即拒绝（fail-closed）。对象/链接未在 `policies.json` 声明则运行时拒绝，声明文件缺失则全拒，`schema_version != 2` 在装载期硬失败（`ValueError`）。这一原则在权限面有三个具体强制点：`policies.json` 缺失时 `object_policies={}` 且 `_missing_file=True`，结果是对象级鉴权全拒；装载期 `schema_version != 2` 直接 `ValueError` 中止；声明角色集若不在 `ROLE_RANK` 内同样 `ValueError`。解决的问题是任何未显式授权的访问都不被默认放行，杜绝“配置漏写=悄悄放行”；代价是任何声明遗漏都会让对应能力整体不可用，而非降级放行——这是刻意选择的保守默认。

写路径唯一。所有写操作经 `ActionExecutor`；Function 只读，SQL 白名单强制首词为 `SELECT` / `WITH`，`_FORBIDDEN_SQL` 正则命中任何写关键字即拒。在 `ActionExecutor` 内部，一次状态迁移要过五步校验（角色占位符拦截、必填 `legal_basis`、状态机合法性、显式 `only_from` 收紧、权限上下文一致性），并经两阶段提交（submit 幂等键 `f"{action}:{clue}:{sha256(...)[:16]}"` → `approve` 必须具名 → `dispatch`，未 `approve` 抛 `NotApprovedError`）。解决的问题是写行为的可审计、可审批、可幂等；代价是任何新写能力都必须挂到 `ActionExecutor` 上，无法在旁路随意落库。